PDF Tool — Re-Audit (Current State)

Reassessment after the full remediation programme · 14 Jul 2026

Live: pdftool-web.onrender.com · Repo: github.com/mukeshroyhub/pdf_tool · Status: **LIVE**

1. Executive summary

The original audit rated the app **~60%** production-ready: strong code, but blocked by ephemeral file storage, an expiring database, no CI, security gaps, and free-tier crashes. Every one of those blockers has been resolved. The app is now **genuinely production-grade** — secured, with durable data and files, automated testing, working transactional email, three auth methods (email, Google, guest), and a polished UI (dark mode, tool grid, branded 404). **Updated readiness: ~85%.** The only remaining code-level risk is the database being synced with `db push` on every boot; the rest are free-tier limits and optional hardening.

2. Original blockers — all resolved

Original finding (was)	Now
Uploaded files lost on restart (ephemeral disk)	Fixed — Supabase S3 object storage
Free Postgres expires in ~30 days	Fixed — migrated to Neon (no expiry)
No lockfile, no CI	Fixed — lockfile committed + GitHub Actions CI
Shared global guest account (data leak)	Fixed — isolated per-session guests + auto-cleanup
Security findings S1–S7	Fixed — all addressed (see §4)
multer 1.x (DoS CVEs)	Fixed — upgraded to multer 2.x
Email verification impossible (no SMTP)	Fixed — Brevo HTTPS API; verification works
OCR crashes the API (out of memory)	Fixed — OCR/Office removed; image 1.3 GB lighter
Sleeps after 15 min → 502s	Mitigated — UptimeRobot keeps API awake

3. Production-readiness scorecard

Dimension	Was	Now	Note
Reliability / durability	30%	90%	Neon + Supabase; data & files survive restarts
Security	75%	90%	All findings fixed; OAuth + email hardened
Maintainability	85%	90%	CI, tests, clean architecture, documentation
Stability	70%	80%	OCR crash gone; cold start mitigated by pinger
Deployment quality	60%	75%	Still 'db push' on boot; occasional manual deploy
Scalability	55%	65%	Lighter image; still single small instance, no queue
OVERALL	~60%	~85%	Production-grade for real use

4. Security review (current)

- No hardcoded secrets in the repo; .env is git-ignored; all credentials are environment variables set in the host dashboard (verified).
- Auth: bcrypt (cost 12), rotating hashed refresh tokens with reuse-detection, httpOnly/Secure/SameSite cookies, timing-safe login, enumeration-safe reset.

- Google OAuth uses a state cookie (CSRF), server-side code exchange, id-token audience check, and a single-use handoff code — proxy-safe.
- Uploads validated by magic bytes (not just client MIME); path-traversal blocked; strict rate limiting; helmet + CSP; HSTS on the web app.
- Email over HTTPS (Brevo) with clean error handling — a mail failure can no longer crash the API.

5. What remains

One real risk (worth fixing):

- The API runs `prisma db push --accept-data-loss` on every startup. Harmless while the schema is stable, but a future schema change could drop data. Switching to committed **Prisma migrations** removes this risk — the single most valuable remaining code change.

Optional hardening & features (not blocking):

- Compiled API build (ship dist/ instead of running tsx); response compression; request logging.
- Web / end-to-end tests (API tests are solid; the frontend has none).
- Accessibility pass (button labels, colour contrast).
- Feature growth: password-protect/unlock, e-signatures, page numbers, ZIP download.

Free-tier reality (not defects):

- One small Render instance; the keep-alive pinger consumes most of the 750 free instance-hours/month — watch the meter.
- For real traffic or to remove sleep entirely, move compute to a small VPS or Oracle Cloud (deploy guide already in the repo).

6. Housekeeping (do soon)

- Revoke the old GitHub personal-access tokens.
- Rotate the Neon database password and the Brevo SMTP key (both appeared in screenshots during setup).

Verdict: a real, working, polished product. From ~60% to ~85% readiness. Recommended final step: Prisma migrations.